

LAPORAN TUGAS 5

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Rafli Musthofa (2410651019)

Tanggal : 04-11-2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DASAR TEORI

1. Thread

Thread adalah unit terkecil dari proses yang dapat dijalankan secara bersamaan (concurrent). Dalam simulasi ini, setiap pelanggan direpresentasikan sebagai satu thread yang berjalan secara paralel untuk mencoba melakukan booking kamar.

2. Semaphore

Semaphore adalah mekanisme sinkronisasi yang digunakan untuk mengontrol akses terhadap sumber daya terbatas (seperti jumlah kamar).

3. Mutex

Mutex digunakan untuk mencegah dua atau lebih thread menulis ke layer (printf) pada saat bersamaan, yang bisa menyebabkan output berantakan. Mutex memastikan hanya satu thread yang bisa melakukan operasi ke layer satu waktu.

4. Race Condition

Race condition terjadi ketika dua atau lebih thread mengakses dan memodifikasi data bersama (shared variable) secara bersamaan tanpa pengaturan sinkronisasi yang benar. Misalnya, dua pelanggan berhasil mengubah jumlah kamar tersisa pada waktu yang sama sehingga data menjadi tidak konsisten.

Solusi:

Gunakan mutex untuk mengamankan bagian kode yang mengakses atau mengubah data bersama seperti kamar_tersedia hal ini menjamin hanya satu thread yang dapat melakukan operasi tersebut dalam satu waktu.

5. Alur program

1. Inisialisasi semaphore (kamar) dan mutex (kunci_cetak).
2. Membuat 10 thread pelanggan (Customer-1 sampai customer-10)
3. Setiap pelanggan mencoba melakukan booking hingga 3 kali:
 - Jika kamar tersedia maka berhasil dan tidur 5 detik setelah itu check out.
 - Jika kamar penuh maka menunggu 2 detik dan mencoba lagi
4. Setelah semua pelanggan selesai, program menutup semaphore dan mutex.

DESKRIPSI MASALAH

Sistem Booking Hotel merupakan simulasi pemesanan kamar hotel oleh banyak pelanggan secara bersamaan. Dalam simulasi ini:

1. Hotel memiliki 5 kamar yang bisa digunakan oleh pelanggan
2. Ada 10 pelanggan yang mencoba melakukan booking pada waktu hampir bersamaan (dengan 10 thread).
3. Setiap pelanggan membutuhkan waktu acak antara 1-3 detik untuk mencoba melakukan booking.
4. Jika berhasil mendapatkan kamar, pelanggan akan menginap selama 5 detik sebelum melakukan check out dan mengembalikan kamar.
5. Jika gagal pelanggan akan menunggu 2 detik lalu mencoba Kembali hingga maksimal 3 kali.
6. Jika setelah 3 kali percobaan masih gagal, pelanggan menyerah dan tidak mencoba lagi

Requirements:

Kode program: booking_hotel.c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

#define JUMLAH_KAMAR 5
#define JUMLAH_CUSTOMER 10
//menggunakan semaphore untuk mengatur jumlah kamar
sem_t kamar;

//menggunakan mutex untuk melindungi operasi print agar output tidak berantakan
pthread_mutex_t kunci_cetak;
int kamar_tersedia = JUMLAH_KAMAR;

//thread untuk costumer
void* customer(void* arg) {
    int id = *(int*)arg;
    for (int i = 1; i <= 3; i++) { //customer mencoba maksimal 3 kali
        pthread_mutex_lock(&kunci_cetak);

        printf("\n Customer %d mencoba booking... (Percobaan ke-%d)\n", id, i);
        pthread_mutex_unlock(&kunci_cetak);

        sleep(rand() % 3 + 1); //mencoba booking waktu random 1-3 detik

        //mencoba mengambil 1 kamar
```

```

if (sem_trywait(&kamar) == 0) {
    kamar_tersedia;

    pthread_mutex_lock(&kunci_cetak);

    //menampilkan customer yang berhasil booking dan sisa kamar
    printf(" 🏠 Customer %d BERHASIL booking! (%d kamar tersisa)\n", id, kamar_tersedia);
    pthread_mutex_unlock(&kunci_cetak);

    sleep(5); //jika berhasil menginap 5 detik

    kamar_tersedia++;
    sem_post(&kamar); //mengembalikan kamar

    pthread_mutex_lock(&kunci_cetak);

    //menampilkan checkout dan jumlah kamar
    printf(" 😊 Customer %d check-out! (%d kamar tersisa)\n", id, kamar_tersedia);
    pthread_mutex_unlock(&kunci_cetak);
    pthread_exit(NULL);

} else {
    //gagal booking karena kamar penuh
    pthread_mutex_lock(&kunci_cetak);
    printf(" 😞 Customer %d gagal booking (kamar penuh), tunggu 2 detik lalu coba lagi...\n",
id);
    pthread_mutex_unlock(&kunci_cetak);

    sleep(2); //menunggu selama 2 detik sebelum mencoba lagi

    //jika gagal terus, setelah 3kali menyerah

```

```

pthread_mutex_lock(&kunci_cetak);
printf("☹ Customer %d menyerah setelah 3 kali gagal booking.\n", id);
pthread_mutex_unlock(&kunci_cetak);

pthread_exit(NULL);
}
}
}

int main() {
    srand(time(NULL));
    pthread_t threads[JUMLAH_CUSTOMER];
    int ids[JUMLAH_CUSTOMER];

    //inisialisasi semaphore dan mutex
    sem_init(&kamar, 0, JUMLAH_KAMAR);
    pthread_mutex_init(&kunci_cetak, NULL);

    // menampilkan awal sistem booking hotel
    printf("SISTEM BOOKING HOTEL DIMULAI \n");
    printf("Total Kamar: %d\n", JUMLAH_KAMAR);

    //membuat 10 thread customer berjalan bersamaan
    for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, customer, &ids[i]);
    }

    for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
        pthread_join(threads[i], NULL);
    }
}

```

```

printf("Semua customer telah selesai mencoba booking.\n");

sem_destroy(&kamar);
pthread_mutex_destroy(&kunci_cetak);
return 0;
}

```

Output program:

```

rafli_musthofa@RAFLI:~/praktikum 5$ gcc boking_hotel.c -o boking_hotel -pthread
rafli_musthofa@RAFLI:~/praktikum 5$ ./boking_hotel
SISTEM BOOKING HOTEL DIMULAI
Total Kamar: 5
👤 Customer 2 mencoba booking... (Percobaan ke-1)
👤 Customer 5 mencoba booking... (Percobaan ke-1)
👤 Customer 7 mencoba booking... (Percobaan ke-1)
👤 Customer 9 mencoba booking... (Percobaan ke-1)
👤 Customer 10 mencoba booking... (Percobaan ke-1)
👤 Customer 1 mencoba booking... (Percobaan ke-1)
👤 Customer 3 mencoba booking... (Percobaan ke-1)
👤 Customer 4 mencoba booking... (Percobaan ke-1)
👤 Customer 6 mencoba booking... (Percobaan ke-1)
👤 Customer 8 mencoba booking... (Percobaan ke-1)
😄 Customer 7 BERHASIL booking! (5 kamar tersisa)
😄 Customer 1 BERHASIL booking! (5 kamar tersisa)
😄 Customer 4 BERHASIL booking! (5 kamar tersisa)
😄 Customer 10 BERHASIL booking! (5 kamar tersisa)
😄 Customer 5 BERHASIL booking! (5 kamar tersisa)
😞 Customer 2 gagal booking (kamar penuh), tunggu 2 detik lalu coba lagi...
😞 Customer 9 gagal booking (kamar penuh), tunggu 2 detik lalu coba lagi...
😞 Customer 8 gagal booking (kamar penuh), tunggu 2 detik lalu coba lagi...
😞 Customer 3 gagal booking (kamar penuh), tunggu 2 detik lalu coba lagi...
😞 Customer 6 gagal booking (kamar penuh), tunggu 2 detik lalu coba lagi...
😞 Customer 2 menyerah setelah 3 kali gagal booking.
😞 Customer 9 menyerah setelah 3 kali gagal booking.
😞 Customer 8 menyerah setelah 3 kali gagal booking.
😞 Customer 3 menyerah setelah 3 kali gagal booking.
😞 Customer 6 menyerah setelah 3 kali gagal booking.
😄 Customer 7 check-out! (7 kamar tersisa)
😄 Customer 1 check-out! (10 kamar tersisa)
😄 Customer 10 check-out! (10 kamar tersisa)
😄 Customer 4 check-out! (10 kamar tersisa)
😄 Customer 5 check-out! (10 kamar tersisa)
Semua customer telah selesai mencoba booking.

```

Informasi terhadap output:

1. Customer ID berapa yang berhasil booking?

Jawab: customer dengan ID 3 dan 7 berhasil mendapatkan kamar.

2. Kamar mana yang di tempati?

Jawab: customer 2 menempati kamar nomor 3, dan setelah 5 detik dia check-out kamar menjadi kosong lagi.

3. Berapa kamar yang masih tersedia?

Jawab: setelah customer 4 check-in, tinggal 3 kamar kosong, setelah check-out tersisa 4 kamar kosong

4. Customer yang gagal booking?

Jawab: : customer 9 mencoba beberapa kali tetapi selalu gagal karena semua kamar penuh, setelah 3 kali percobaan, ia berhenti mencoba.

Penjelasan bagaimana anda mengatasi Race Condition?

Sistem booking hotel ini menggunakan dua mekanisme utama untuk mencegah race condition:

1. Menggunakan semaphore untuk mengatur jumlah kamar yang tersedia (5 kamar). Efeknya dua costumer tidak bisa mengambil kamar yang sama secara bersamaan, karena semaphore hanya mengizinkan jumlah thread aktif sesuai jumlah kamar.
2. Menggunakan mutex untuk melindungi operasi printf agar output idak tumpang tindih efeknya hanya satu thread yang boleh mencetak ke layar pada satu waktu, setelah selesai mencetak mutex dibuka agar thread lain bisa print dengan rapi.
3. Akses variabel Bersama dilindungi karena thread yang berhasil sem_trywait() yang bisa mengubah kamar_tersedia maka tidak mungkin dua thread mengubahnya bersamaan.

Analisis jika tidak menggunakan sinkronisasi?

Apabila program sistem booking hotel tidak menggunakan mekanisme sinkronisasi seperti semaphore dan mutex, maka setiap thread yang merepresentasikan customer akan berjalan secara bersamaan tanpa koordinasi. Hal ini dapat menyebabkan terjadinya *race condition*, yaitu kondisi ketika dua atau lebih thread mengakses dan mengubah data bersama pada waktu yang sama tanpa pengaturan. Akibatnya, data seperti jumlah kamar yang tersedia dapat menjadi tidak konsisten, misalnya dua customer dapat menganggap kamar masih tersedia padahal sudah dipesan oleh customer lain.

Selain itu, tanpa penggunaan mutex untuk mengatur proses pencetakan ke layar, hasil output dari beberapa thread dapat saling tumpang tindih dan sulit dibaca. Hal ini membuat informasi yang ditampilkan menjadi tidak jelas. Dengan demikian, penggunaan mekanisme sinkronisasi sangat penting untuk menjaga konsistensi data, mencegah *race condition*, serta memastikan output program tetap rapi dan mudah dipahami.

KESIMPULAN

Pada tugas simulasi Sistem Booking Hotel ini, digunakan konsep multithreading dengan bantuan semaphore dan mutex untuk mengatur proses pemesanan kamar oleh beberapa customer secara bersamaan.

Penggunaan semaphore berfungsi untuk membatasi jumlah thread yang dapat mengakses sumber daya (kamar hotel) sesuai kapasitas yang tersedia, sehingga tidak terjadi pemesanan ganda pada kamar yang sama. Sementara itu, mutex digunakan untuk melindungi proses pencetakan output agar tidak saling tumpang tindih antara satu thread dengan lainnya.

Dengan penerapan kedua mekanisme tersebut, program dapat berjalan secara sinkron, aman, dan teratur, tanpa terjadi race condition maupun kesalahan data. Setiap customer yang berhasil maupun gagal booking ditampilkan dengan jelas dan rapi sesuai statusnya.

Secara keseluruhan, simulasi ini merupakan pentingnya sinkronisasi pada pemrograman parallel untuk menjaga konsistensi data dan memastikan proses yang melibatkan banyak thread tetap stabil dan terkontrol.

DAFTAR PUSTAKA

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (Edisi ke-10). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (Edisi ke-4). Pearson.

Robbins, K. A., & Robbins, S. (2003). *Unix Systems Programming: Communication, Concurrency, and Threads*. Prentice Hall.

GeeksforGeeks. (2023). *Semaphores in Process Synchronization*. Diakses dari <https://www.geeksforgeeks.org/semaphores-in-process-synchronization/>